

Проверка гипотез в бизнесе

В этом проекте мы анализируем крупный сервис проката самокатов GoFast. Компания предоставила три датасета с данными о пользователях сервиса, их поездках за один год, а также стоимости услуг в подписках.

Наша цель – проанализировать демографию пользователей и особенности использования самокатов, а также определить возможную выгоду от распространения платной подписки на самокаты.

Описание данных

Таблица с пользователями `users_go.csv`

- `user_id` — уникальный идентификатор пользователя.
- `name` — имя пользователя.
- `age` — возраст.
- `city` — город.
- `subscription_type` — тип подписки: `free`, `ultra`.

Таблица с поездками `rides_go.csv`

- `user_id` — уникальный идентификатор пользователя.
- `distance` — расстояние в метрах, которое пользователь проехал в текущей сессии.
- `duration` — продолжительность сессии в минутах, то есть время с того момента, как пользователь нажал кнопку «Начать поездку», до того, как он нажал кнопку «Завершить поездку».
- `date` — дата совершения поездки.

Таблица с подписками `subscriptions_go.csv`

- `subscription_type` — тип подписки.
- `minute_price` — стоимость одной минуты поездки по этой подписке.
- `start_ride_price` — стоимость начала поездки.
- `subscription_fee` — стоимость ежемесячного платежа.

Задание 1. Загрузка данных

Задача 1.1 Импорт библиотеки pandas

```
In [1]: import pandas as pd
```

Задача 1.2 Чтение данных

```
In [2]: df_users_go = pd.read_csv('users_go.csv')
df_rides_go = pd.read_csv('rides_go.csv')
df_subscriptions_go = pd.read_csv('subscriptions_go.csv')
```

Задача 1.3 Печать первых строк

Выведем первые пять строк каждого датафрейма. Познакомимся с содержанием таблиц.

```
In [3]: display(df_users_go.head())
display(df_rides_go.head())
display(df_subscriptions_go.head())
```

	user_id	name	age	city	subscription_type
0	1	Кира	22	Тюмень	ultra
1	2	Станислав	31	Омск	ultra
2	3	Алексей	20	Москва	ultra
3	4	Константин	26	Ростов-на-Дону	ultra
4	5	Адель	28	Омск	ultra

	user_id	distance	duration	date
0	1	4409.919140	25.599769	2021-01-01
1	1	2617.592153	15.816871	2021-01-18
2	1	754.159807	6.232113	2021-04-20
3	1	2694.783254	18.511000	2021-08-11
4	1	4028.687306	26.265803	2021-08-28

	subscription_type	minute_price	start_ride_price	subscription_fee
0	free	8	50	0
1	ultra	6	0	199

Задача 1.4 Подсчёт строк

Определим количество строк в каждом из трёх датафреймов. Это поможет оценить объём данных и выявить аномалии, например слишком малое количество записей.

```
In [4]: print(df_users_go.shape[0], df_rides_go.shape[0], df_subscriptions_go.shape[0])

1565 18068 2
```

Задание 2. Знакомство с данными и их предварительная подготовка

Прежде чем начать анализ, важно убедиться в качестве предоставленных данных. Выполним несколько шагов, чтобы лучше понять структуру и устранить потенциальные проблемы в таблицах.

Задача 2.1 Определение типов данных

В датафрейме `df_rides_go` выведем типы данных каждой колонки стандартным атрибутом `dtypes`. Так мы проверим корректность типов и подготовим данные для дальнейшей обработки.

```
In [5]: df_rides_go.dtypes
```

```
Out[5]: user_id      int64
distance  float64
duration  float64
date      object
dtype: object
```

Задача 2.2 Преобразование формата даты

Приведем столбец `date` в датафрейме `df_rides_go` к типу даты pandas, используя метод `pd.to_datetime()`. Этот шаг упростит работу с временными данными.

```
In [6]: df_rides_go['date'] = pd.to_datetime(df_rides_go['date'])
```

Задача 2.3 Создание столбца с месяцем

На основе столбца `date` создаем новый столбец `month`, содержащий номер месяца. Это нужно для последующей группировки данных и анализа сезонных трендов.

```
In [7]: df_rides_go['month'] = df_rides_go['date'].dt.month
```

Задача 2.4 Поиск дублей и пропусков

В датафрейме пользователей `df_users_go` определим количество пропусков и дубликатов. Это необходимо для понимания полноты и уникальности данных пользователей.

```
In [8]: print(df_users_go.isna().sum().sum(), df_users_go.duplicated().sum())
0 31
```

Задача 2.5 Заполнение пропусков и удаление дублей

При наличии пропущенных значений заполним их нулями, а все дубликаты удалим. Это шаг необходим для улучшения качества данных, что является ключевым для

точного анализа.

```
In [9]: df_users_go = df_users_go.fillna(0)
df_users_go = df_users_go.drop_duplicates()
```

Задача 2.6 Округление длительности поездки

В датафрейме поездок `df_rides_go` округлим время поездки `duration` до целого числа стандартным методом `round()`. Затем приведем эту колонку к целочисленному типу `int`. Этот шаг затем поможет нам правильно рассчитать прибыль, так как плата взимается только за целое число минут.

```
In [10]: df_rides_go['duration'] = df_rides_go['duration'].round().astype(int)
```

Задание 3. Исследовательский анализ данных (EDA)

Задача 3.1 Импорт библиотеки matplotlib

Импортируем библиотеку `matplotlib.pyplot` с сокращением `plt`.

```
In [11]: import matplotlib.pyplot as plt
```

Задача 3.2 Количество пользователей по городам.

Пользуясь данными из датафрейма `df_users_go`, изучим количество пользователей в каждом из городов. Создадим и выведем на экран объект `pd.Series`, названный `users_by_city_count`, с одной колонкой `count` — суммарным количеством пользователей в каждом городе `city`, отсортированным по убыванию.

```
In [12]: users_by_city_count = df_users_go['city'].value_counts()
print(users_by_city_count)
```

```
Пятигорск      219
Екатеринбург   204
Ростов-на-Дону  198
Краснодар      193
Сочи           189
Омск           183
Тюмень         180
Москва         168
Name: city, dtype: int64
```

Задача 3.3 Количество пользователей подписки

Создадим и выведем на экран серию `subscription_type_count` с количеством пользователей для каждого типа подписки `subscription_type`.

```
In [13]: subscription_type_count = df_users_go['subscription_type'].value_counts()
subscription_type_count
```

```
Out[13]: free      835
ultra      699
```

Name: subscription_type, dtype: int64

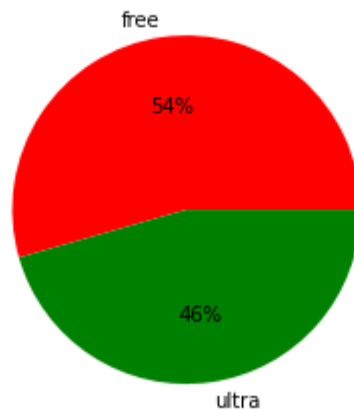
Задача 3.4 Круговая диаграмма

Построим круговую диаграмму, указав процент пользователей с `free` и `ultra` подпиской. Вызовем метод `plot()` для серии `subscription_type_count` с указанием нужного аргумента для построения круговой диаграммы.

```
In [14]: subscription_type_count.plot(
          kind='pie',
          title='Соотношение пользователей с подпиской и без подписки',
          autopct='%.0f%%',
          ylabel='',
          colors=['red', 'green'])

plt.show()
```

Соотношение пользователей с подпиской и без подписки



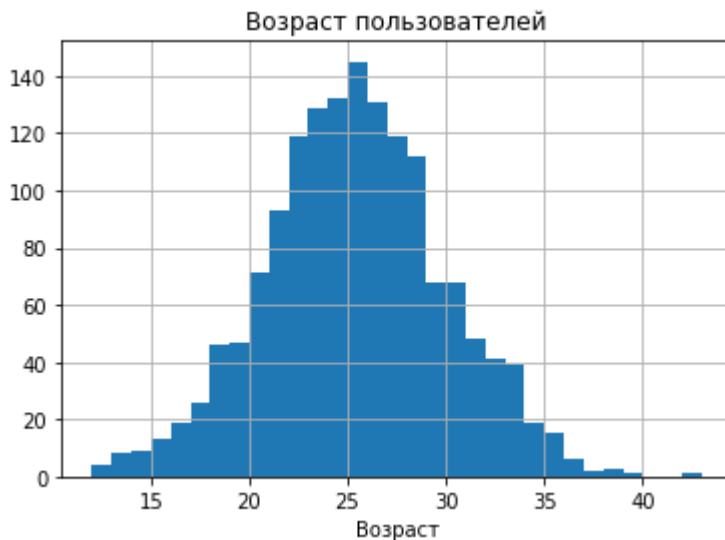
Задача 3.5 Гистограмма возрастов

Построим гистограмму возрастов `age` пользователей самокатов.

```
In [15]: n_bins = df_users_go['age'].max() - df_users_go['age'].min()

df_users_go['age'].hist(bins=n_bins)
plt.title('Возраст пользователей')
plt.xlabel('Возраст')

plt.show()
```



Задача 3.6 Несовершеннолетние пользователи

Рассчитаем долю несовершеннолетних (возрастом менее 18 лет) пользователей самокатов.

```
In [16]: users_under_18_ratio = int((df_users_go[df_users_go['age'] < 18].shape[0] / df_users_go.shape[0]) * 100)
print(f'Доля несовершеннолетних пользователей самокатов составляет {users_under_18_ratio}%.')
```

Доля несовершеннолетних пользователей самокатов составляет 5%.

Задача 3.7 Характеристики длительности поездки

Длительность поездки является важной метрикой в работе сервиса проката самокатов. Если средняя длительность поездок будет слишком высокой, самокаты будут быстрее выходить из строя. Если слишком низкой, значит, клиентам что-то не нравится в сервисе.

В этом задании с помощью описательной статистики изучим длительность поездок:

```
In [17]: duration_mean = int(round(df_rides_go['duration'].mean()))
duration_std = int(round(df_rides_go['duration'].std()))

duration_pct25 = int(round(df_rides_go['duration'].quantile(0.25)))
duration_pct75 = int(round(df_rides_go['duration'].quantile(0.75)))

print(f'Средняя длительность поездки {duration_mean} минут со стандартным отклонением {duration_std} минут. 25-й перцентиль равен {duration_pct25} минут, 75-й перцентиль равен {duration_pct75} минут.')
```

Средняя длительность поездки 18 минут со стандартным отклонением 6. Основная часть поездок занимает от 14 до 22 минут.

Задание 4. Объединение данных

Объединим 3 таблицы в один датафрейм.

Задача 4.1 Объединение таблицы пользователей и поездок

Объединим датафреймы с информацией о пользователях `df_users_go` и поездках `df_rides_go`. Полученный датафрейм назовем просто `df`.

```
In [18]: df = df_users_go.merge(df_rides_go, on='user_id', how='left')
```

Задача 4.2 Присоединение информации о подписках

Присоединим к полученному датафрейму `df` информацию о подписках из `df_subscriptions_go`, используя метод `merge()` по общей колонке и тип объединения `left`.

```
In [19]: df = df.merge(df_subscriptions_go, on='subscription_type', how='left')
```

Задача 4.3. Размеры объединённого датафрейма

Для финальной проверки выведем на экран первые пять строк датафрейма, а также количество строк и столбцов полученного датафрейма `df`. Этот этап позволит убедиться, что никакие записи не потерялись при объединении.

```
In [20]: # Выводим первые строки датафрейма
display(df.head())

# Выводим количество строк и столбцов в объединённом датафрейме
n_rows, n_cols = df.shape
print(f'В полученном датафрейме {n_rows} строк и {n_cols} столбцов.')
```

	user_id	name	age	city	subscription_type	distance	duration	date	month	min
0	1	Кира	22	Тюмень	ultra	4409.919140	26	2021-01-01	1	
1	1	Кира	22	Тюмень	ultra	2617.592153	16	2021-01-18	1	
2	1	Кира	22	Тюмень	ultra	754.159807	6	2021-04-20	4	
3	1	Кира	22	Тюмень	ultra	2694.783254	19	2021-08-11	8	
4	1	Кира	22	Тюмень	ultra	4028.687306	26	2021-08-28	8	

В полученном датафрейме 18068 строк и 12 столбцов.

Задача 4.4 Отдельные датафреймы для пользователей с подпиской и без

Создадим два вспомогательных датафрейма на основе `df`: первый только для пользователей с подпиской `df_ultra` и второй только для пользователей без подписки `df_free`.

```
In [21]: df_ultra = df[df['subscription_type'] == 'ultra']
```

```
df_free = df[df['subscription_type'] == 'free']
```

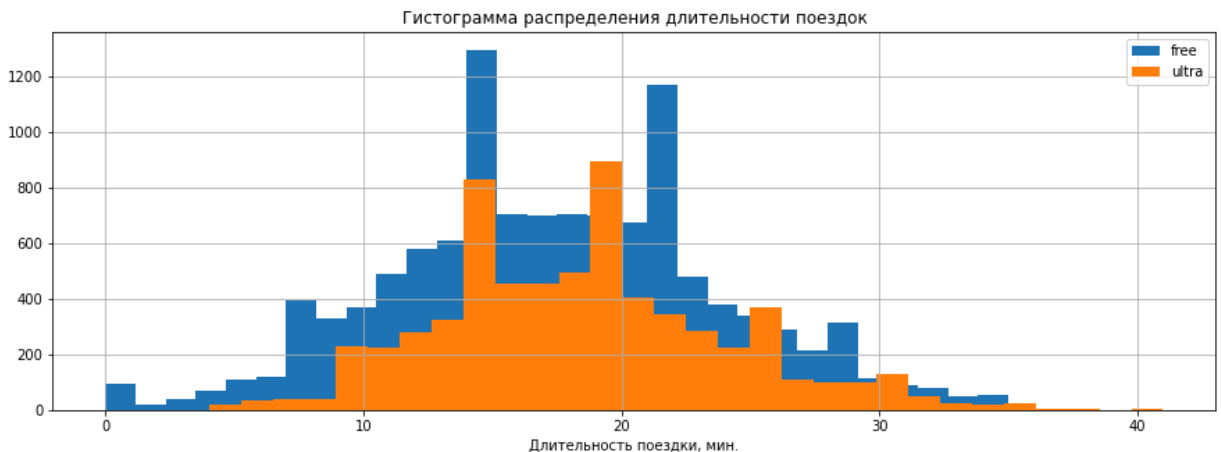
Задача 4.5 Гистограмма длительности поездок для обеих групп

На одном графике построим гистограмму распределения длительности поездок `duration` для пользователей с подпиской и без.

In [22]:

```
# Гистограмма длительности поездки для пользователей с подпиской и без
plt.figure(figsize=(15, 5))
df_free['duration'].hist(bins=30, label='free')
df_ultra['duration'].hist(bins=30, label='ultra')
plt.xlabel('Длительность поездки, мин.')
plt.title('Гистограмма распределения длительности поездок')
plt.legend()
plt.show()

# Расчет и вывод на экран средней длительности поездки для пользователей с
mean_duration_free = round(df_free['duration'].mean())
mean_duration_ultra = round(df_ultra['duration'].mean())
print(f'Средняя длительность поездки для пользователей без подписки {mean_dur
```



Средняя длительность поездки для пользователей без подписки 17 мин, а для пользователей с подпиской 19 мин

Задание 5. Подсчёт выручки

Данные о количестве и длительности поездок объединены с ценами и тарифами, а значит, теперь есть всё необходимое, чтобы рассчитать важный для бизнеса показатель — выручку.

Задача 5.1 Данные сгруппированные по нужным колонкам

Сгруппируем данные по следующим столбцам: `user_id`, `name`, `subscription_type`, `month`. Группировку производим в датафрейме `df_gp`.

In [23]:

```
df_gp = df.groupby(['user_id', 'name', 'subscription_type', 'month'], as_index=False)
```

Задача 5.2 Подсчёт агрегированных метрик

Создадим новый датафрейм `df_agg`. В нём для каждой группы рассчитаем и добавим в датафрейм следующие агрегированные значения, используя метод `agg()`:

- `total_distance` — сумма значений в столбце `distance`.
- `total_duration` — сумма значений в столбце `duration`.
- `rides_count` — количество элементов в столбце `duration`.
- `subscription_type` — первое значение в группе для столбца `subscription_type`.
- `minute_price` — первое значение в группе для столбца `minute_price`.
- `start_ride_price` — первое значение в группе для столбца `start_ride_price`.
- `subscription_fee` — первое значение в группе для столбца `subscription_fee`.

In [24]:

```
df_agg = df_gp.agg(  
    total_distance=('distance', 'sum'),  
    total_duration=('duration', 'sum'),  
    rides_count=('duration', 'count'),  
    subscription_type=('subscription_type', 'first'),  
    minute_price=('minute_price', 'first'),  
    start_ride_price=('start_ride_price', 'first'),  
    subscription_fee=('subscription_fee', 'first'),  
)
```

Задача 5.3 Функция для подсчёта выручки

Создадим функцию `calculate_monthly_revenue(row)` для расчёта месячной выручки по формуле:

$$\text{monthly_revenue} = \text{start_ride_price} \times \text{rides_count} + \text{minute_price} \times \text{total_duration} + \text{subscription_fee}$$

В качестве входных данных функция будет принимать одну строку `row` датафрейма. Вспомним, как в таком случае обращаться к каждой колонке.

Здесь:

- `start_ride_price * rides_count` — выручка от начала каждой поездки.
- `minute_price * total_duration` — выручка за время использования.
- `subscription_fee` — фиксированная выручка от подписок.

In [25]:

```
def calculate_monthly_revenue(row):  
    return row['start_ride_price'] * row['rides_count'] + row['minute_price']
```

Задача 5.4 Создание столбца с месячной выручкой на пользователя

Создадим новый столбец с месячной выручкой на пользователя `monthly_revenue`. Для этого применим функцию `calculate_monthly_revenue(row)` к каждой строке агрегированного датафрейма `df_agg`.

```
In [26]: df_agg['monthly_revenue'] = df_agg.apply(calculate_monthly_revenue, axis=1)
```

Задача 5.5 Поиск пользователя с максимальной выручкой

Исследуем полученные значения выручки. Найдем пользователя с максимальной суммарной выручкой за весь период наблюдения и для него выведем на экран

`user_id`, имя, месяц, количество поездок и выручку за каждый из месяцев.

```
In [27]: user_max_revenue = df_agg.groupby('user_id')['monthly_revenue'].sum().idxmax()

print(
    df_agg[df_agg['user_id'] == user_max_revenue][
        ['user_id', 'name', 'month', 'rides_count', 'monthly_revenue']
    ]
)
```

	user_id	name	month	rides_count	monthly_revenue
8877	1236	Александр	1	2	228
8878	1236	Александр	2	3	614
8879	1236	Александр	3	5	762
8880	1236	Александр	4	1	202
8881	1236	Александр	5	3	574
8882	1236	Александр	6	1	282
8883	1236	Александр	7	1	290
8884	1236	Александр	8	2	452
8885	1236	Александр	9	1	122
8886	1236	Александр	10	3	430
8887	1236	Александр	11	3	494
8888	1236	Александр	12	2	476

Задание 6. Проверка гипотез

Задача 6.1 Импорт библиотеки SciPy

Импортируем библиотеку `scipy.stats` с сокращением `st`.

```
In [28]: import scipy.stats as st
```

Задача 6.2 Вспомогательная функция для интерпретации результатов

Напишем вспомогательную функцию `print_stattest_results(p_value, alpha)`, которая будет интерпретировать результаты статистического теста на основе p-value и заданного уровня значимости (α -уровня). Функция должна решать, следует ли принять альтернативную гипотезу или сохранить нулевую гипотезу.

```
In [29]: def print_stattest_results(p_value: float, alpha: float = 0.05):
    if p_value < alpha:
        print(f'Полученное значение p_value={p_value} меньше критического уро
    else:
        print(f'Полученное значение p_value={p_value} больше критического уро
```

```
print_stattest_results(p_value=0.0001)
print_stattest_results(p_value=0.1)
```

Полученное значение $p_value=0.0001$ меньше критического уровня $\alpha=0.05$. Принимаем альтернативную гипотезу.
Полученное значение $p_value=0.1$ больше критического уровня $\alpha=0.05$. Опровергнуть нулевую гипотезу нельзя.

Задача 6.3 Длительность для пользователей с подпиской и без

Важно понять, тратят ли пользователи с подпиской больше времени на поездки?

Сформулируем нулевую и альтернативную гипотезы:

- Нулевая гипотеза (H_0): Среднее время поездки у пользователей с подпиской и без подписки одинаковое.
- Альтернативная гипотеза (H_1): Среднее время поездки у пользователей с подпиской больше, чем у пользователей без подписки.

In [30]:

```
ultra_duration = df_ultra['duration']
free_duration = df_free['duration']

results = st.ttest_ind(ultra_duration, free_duration, alternative='greater')
p_value = results.pvalue
print_stattest_results(p_value)

ultra_mean_duration = round(df_ultra['duration'].mean(), 2)
free_mean_duration = round(df_free['duration'].mean(), 2)

print(f'Средняя длительность поездки тарифа Ultra {ultra_mean_duration}')
print(f'Средняя длительность поездки тарифа Free {free_mean_duration}')
```

Полученное значение $p_value=3.1600689435611813e-35$ меньше критического уровня $\alpha=0.05$. Принимаем альтернативную гипотезу.
Средняя длительность поездки тарифа Ultra 18.55
Средняя длительность поездки тарифа Free 17.39

Задача 6.4 Длительность поездки: больше или меньше критического значения

Проанализируем ещё одну важную продуктовую гипотезу. Расстояние одной поездки в 3130 метров — оптимальное с точки зрения износа самоката. Можно ли сказать, что расстояние, которое проезжают пользователи с подпиской за одну поездку, меньше 3130 метров?

Сформулируем нулевую и альтернативную гипотезы:

- Нулевая гипотеза (H_0): Средняя дистанция поездки у пользователей с подпиской равна 3130 м.
- Альтернативная гипотеза (H_1): Средняя дистанция поездки у пользователей с подпиской больше 3130 м.

In [31]:

```
null_hypothesis = 3130
ultra_distance = df_ultra['distance']

results = st.ttest_1samp(ultra_distance, null_hypothesis, alternative='greater')
```

```
p_value = results.pvalue
print_stattest_results(p_value)
```

Полученное значение $p_value=0.9195368847849785$ больше критического уровня $\alpha=0.05$. Опровергнуть нулевую гипотезу нельзя.

Задача 6.5 Прибыль от пользователей с подпиской и без

Проверим гипотезу о том, что выручка от пользователей с подпиской выше, чем выручка от пользователей без подписки.

Сформулируем нулевую и альтернативную гипотезы:

- Нулевая гипотеза (H_0): Средняя месячная выручка у пользователей с подпиской и без подписки одинаковая.
- Альтернативная гипотеза (H_1): Средняя месячная выручка у пользователей с подпиской выше, чем у пользователей без подписки.

In [32]:

```
revenue_ultra = df_agg[df_agg['subscription_type'] == 'ultra']['monthly_revenue']
revenue_free = df_agg[df_agg['subscription_type'] == 'free']['monthly_revenue']

results = st.ttest_ind(revenue_ultra, revenue_free, alternative='greater')
p_value = results.pvalue
print_stattest_results(p_value)

mean_revenue_ultra = round(revenue_ultra.mean())
mean_revenue_free = round(revenue_free.mean())

print(f'Средняя выручка подписчиков Ultra {mean_revenue_ultra} руб')
print(f'Средняя выручка подписчиков Free {mean_revenue_free} руб')
```

Полученное значение $p_value=1.7274069878387966e-37$ меньше критического уровня $\alpha=0.05$. Принимаем альтернативную гипотезу.
Средняя выручка подписчиков Ultra 359 руб
Средняя выручка подписчиков Free 322 руб

Задание 7. Распределения

В компании возникла идея предлагать дополнительную скидку подписчикам, совершающим длительные поездки продолжительностью более 30 минут. Вас просят оценить долю таких поездок.

Задача 7.1 Расчёт выборочного среднего и стандартного отклонения

1. Расчитаем среднюю длительность поездки и сохраним в переменную `mu`.
2. Вычислим стандартное отклонение длительности `duration` и сохраним в переменную `sigma`. Для вычисления стандартного отклонения можно применить метод `std()` к столбцу датасета.
3. Зададим значение переменной `target_time`, равное `30`. Эта переменная будет использоваться для последующего вычисления вероятности.
4. Выведем текстовый вывод по шаблону. При этом округлим оба значения до одного знака после точки стандартной функцией `round()`.

In [33]:

```
# Вычисляем среднее значение
mu = df_ultra['duration'].mean()

# Вычисляем стандартное отклонение
sigma = df_ultra['duration'].std()

# Задаём целевое время
target_time = 30

# Делаем вывод
print(f'Средняя длительность поездки {round(mu, 1)}, стандартное отклонение {
```

Средняя длительность поездки 18.5, стандартное отклонение 5.6.

Задача 7.2 Вычисление значения функции распределения в точке (CDF)

Если вычислить значение функции распределения в точке, это позволит узнать вероятность того, что случайная величина примет значение меньше заданного либо равное ему. Соответственно, если мы хотим ответить на вопрос о вероятности поездки более 30 минут, потребуется использовать CDF.

In [34]:

```
# Вычисляем вероятность того, что случайная величина будет меньше указанного
duration_norm_dist = st.norm(mu, sigma)
prob = round(1 - duration_norm_dist.cdf(target_time), 3) # Используем CDF для
print(f'Вероятность поездки более 30 минут {prob}')
```

Вероятность поездки более 30 минут 0.02

Задача 7.3 Вероятность для интервала (CDF)

Коллеги посчитали, что процент пользователей, для которых будет показана скидка, недостаточно большой и вряд ли поможет в увеличении лояльности клиентов. Дополнительно нас просят проверить, какой процент пользователей совершает поездки в интервале от 20 до 30 минут. Возможно, именно для них стоит провести промоакцию?

In [35]:

```
# Определяем границы интервала
low = 20
high = 30

# Вычисляем вероятность попадания в интервал
prob_interval = round(duration_norm_dist.cdf(high) - duration_norm_dist.cdf(low), 3)

# Выводим результат
print(f'Вероятность того, что пользователь совершит поездку длительностью от
```

Вероятность того, что пользователь совершит поездку длительностью от 20 до 30 минут: 0.377

Задача 7.4 Определение критической дистанции поездок (PPF)

Длительные поездки могут негативно сказываться на сроке службы самоката. В связи с этим принято решение установить критическую дистанцию, превышение которой будет сопровождаться дополнительной платой. Для этого необходимо определить расстояние, которое превышаете только в 10% поездок (90-й процентиль).

Наша задача — смоделировать распределение длительности поездок, предполагая, что оно подчиняется нормальному закону, и рассчитать критическую дистанцию, ниже которой находится 90% всех поездок.

In [36]:

```
# Вычисляем среднее значение
mu = df['distance'].mean()

# Вычисляем стандартное отклонение
sigma = df['distance'].std()

# Вероятность, для которой хотим найти значение (90% случаев)
target_prob = 0.90

# Создаём объект нормального распределения
distance_norm = st.norm(mu, sigma)

# Рассчитываем критическую дистанцию для заданного процентиля поездок
critical_distance = distance_norm.ppf(target_prob)

print(f'{100 * target_prob} % поездок имеют дистанцию ниже критического значе
```

90.0 % поездок имеют дистанцию ниже критического значения 4501.94 М.